

Projeto de Banco de Dados: AVA FICR (Ambiente Virtual de Aprendizagem)

Este documento apresenta a especificação, modelagem e implementação física do banco de dados relacional para o sistema AVA FICR, desenvolvido utilizando o Sistema Gerenciador de Banco de Dados Relacional (SGBDR) Microsoft SQL Server.

1. Objetivo do Projeto

O objetivo deste projeto é projetar, modelar e implementar o banco de dados relacional para um Ambiente Virtual de Aprendizagem (AVA). O sistema visa otimizar a comunicação e gestão acadêmica de uma instituição de ensino superior. Ao término do projeto, os entregáveis consistirão em: * Esquema físico completo e otimizado para o SGBD Microsoft SQL Server. * Carga inicial de dados simulando cenários acadêmicos reais. * Mecanismos de automação em banco de dados (Triggers, Functions, Stored Procedures e Índices) para garantir integridade, consistência e velocidade na recuperação de relatórios.

2. Modelo de Negócio

- Ramo de Atuação:** Educação / Tecnologia Educacional.
- Mercado:** Ensino Superior e Cursos Tecnológicos.
- Cientes/Atores do Sistema:**
 - Alunos (Estudantes):** Utilizam o sistema para visualizar seus quadros de tarefas (Kanban), submeter entregas de atividades, acompanhar suas notas (Boletim) e abrir solicitações administrativas.
 - Professores:** Responsáveis por lançar e gerenciar disciplinas, cadastrar tarefas avaliativas com prazos determinados e lançar notas/feedbacks pedagógicos.
 - Secretaria Acadêmica:** Setor administrativo responsável por gerenciar turmas, matricular alunos em disciplinas, publicar avisos gerais de relevância institucional e decidir/deferir protocolos abertos por estudantes.

3. Levantamento de Requisitos

Abaixo estão descritos os **Requisitos Funcionais (RF)** do banco de dados:

- RF001 - Cadastro de Pessoas:** O sistema deve armazenar informações de Alunos, Professores e Secretários. Cada pessoa possui Nome, E-mail (único), CPF (único) e uma Senha.
- RF002 - Cadastro de Disciplinas:** O sistema deve registrar as disciplinas curriculares informando o nome e associando o professor que a lecionará.
- RF003 - Matrícula de Alunos:** O sistema deve permitir que alunos sejam matriculados em múltiplas disciplinas, gerando uma associação N:N.
- RF004 - Criação de Tarefas:** Professores (ou a Secretaria) podem criar tarefas vinculadas a uma disciplina, definindo título, descrição, prazo (data/hora limite) e pontuação máxima.
- RF005 - Geração do Quadro de Atividades (Kanban):** Toda vez que uma tarefa for criada, o banco de dados deve gerar automaticamente registros de entrega pendentes para todos os alunos matriculados naquela disciplina específica.
- RF006 - Submissão de Entregas:** O aluno pode alterar o status de sua entrega (Pendente, Em Andamento, Entregue) e anexar sua resposta textual.
- RF007 - Lançamento de Notas:** O professor da disciplina deve avaliar as tarefas entregues e lançar notas (que não podem ser negativas nem ultrapassar os pontos máximos da tarefa) e feedback.
- RF008 - Abertura de Protocolos:** Alunos podem abrir solicitações (Prorrogação de Prazo, Trancamento de Disciplina, Emissão de Histórico). A secretaria pode analisar e deferir/indeferir a solicitação.
- RF009 - Mural de Avisos:** A secretaria pode publicar avisos gerais contendo título, conteúdo e data de publicação.

4. Escolha do SGBD

O SGBD escolhido para a realização deste projeto foi o **Microsoft SQL Server**. **Justificativa detalhada:** 1. **Conformidade Corporativa:** É um dos SGBDs líderes de mercado no setor corporativo e educacional, oferecendo alta confiabilidade, estabilidade e suporte a transações ACID complexas. 2. **Linguagem Procedural Robusta (T-SQL):** O Transact-SQL (T-SQL) fornece suporte avançado para o desenvolvimento de Triggers, Stored Procedures de alta performance e funções personalizadas, facilitando o encapsulamento de lógica de negócio no próprio banco de dados. 3. **Ferramentas de Gerenciamento:** O *SQL Server Management Studio (SSMS)* oferece uma suíte completa para análise de performance de consultas, monitoramento de planos de execução de índices e depuração de código procedural. 4. **Segurança de Dados integrada:** Recursos como controle de acesso baseado em papéis (RBAC), auditoria de dados e criptografia nativa.

5. Geração do Modelo Conceitual (DER)

A estrutura do banco de dados conta com **7 entidades** e relacionamentos específicos.

Diagrama Entidade-Relacionamento (Texto Explicativo e Conexões)

- Pessoas** (id, nome, email, cpf, senha, tipo_pessoa)
 - Uma Pessoa pode ser do tipo ALUNO, PROFESSOR ou SECRETARIA.
 - Relaciona-se com Disciplinas (1 Professor leciona N Disciplinas).
 - Relaciona-se com Protocolos (1 Aluno abre N Protocolos).
 - Relaciona-se com Avisos (1 Secretaria cadastra N Avisos).
- Disciplinas** (id, nome, professor_id)
 - Relaciona-se com Pessoas (Tabela de Alunos) de forma N:N (Muitos alunos se matriculam em muitas disciplinas).
 - Relaciona-se com Tarefas (1 Disciplina possui N Tarefas).
- DisciplinaAlunos** (Junction Table N:N)
 - Armazena a matrícula associando aluno_id e disciplina_id.
- Tarefas** (id, titulo, descricao, data_entrega, pontos_maximos, disciplina_id, criado_por_id)
 - Relaciona-se com EntregaTarefas (1 Tarefa gera N registros de entrega - um para cada aluno).
- EntregaTarefas** (id, tarefa_id, aluno_id, status_tarefa, data_entrega, resposta_text, nota, feedback)

- Entidade de ligação que representa o Kanban e as notas dos alunos em cada tarefa.
- **Protocolos** (id, tipo, descricao, status, data_solicitacao, aluno_id, analisado_por_id)
 - Representa solicitações administrativas feitas por alunos.
- **Avisos** (id, titulo, conteudo, data_publicacao, criado_por_id)
 - Representa o mural de avisos da instituição.

6. Geração do Modelo Lógico

O modelo lógico define os tipos de dados e chaves estrangeiras com os domínios nativos do **SQL Server (T-SQL)**:

```
PESSOAS (
    id INT (PK, IDENTITY),
    nome VARCHAR(150) NOT NULL,
    email VARCHAR(100) NOT NULL (UNIQUE),
    cpf VARCHAR(14) NOT NULL (UNIQUE),
    senha VARCHAR(100) NOT NULL,
    tipo_pessoa VARCHAR(20) NOT NULL (CHECK: 'ALUNO', 'PROFESSOR', 'SECRETARIA')
)

DISCIPLINAS (
    id INT (PK, IDENTITY),
    nome VARCHAR(100) NOT NULL,
    professor_id INT (FK -> PESSOAS.id)
)

DISCIPLINA_ALUNOS (
    disciplina_id INT (PK, FK -> DISCIPLINAS.id),
    aluno_id INT (PK, FK -> PESSOAS.id)
)

TAREFAS (
    id INT (PK, IDENTITY),
    titulo VARCHAR(150) NOT NULL,
    descricao TEXT,
    data_entrega DATETIME2 NOT NULL,
    pontos_maximos DECIMAL(5, 2) NOT NULL,
    disciplina_id INT (FK -> DISCIPLINAS.id),
    criado_por_id INT (FK -> PESSOAS.id)
)

ENTREGA_TAREFAS (
    id INT (PK, IDENTITY),
    tarefa_id INT (FK -> TAREFAS.id, ON DELETE CASCADE),
    aluno_id INT (FK -> PESSOAS.id),
    status_tarefa VARCHAR(20) NOT NULL (DEFAULT 'PENDENTE', CHECK: 'PENDENTE', 'EM_ANDAMENTO', 'ENTREGUE'),
    data_entrega DATETIME2,
    resposta_text TEXT,
    nota DECIMAL(4,2) (CHECK: nota >= 0.0 AND nota <= pontos_maximos),
    feedback VARCHAR(250)
)

PROTOCOLOS (
    id INT (PK, IDENTITY(1000, 10)), -- Autonumeração especial iniciada em 1000 incrementando de 10 em 10
    tipo VARCHAR(50) NOT NULL (CHECK: 'PRORROGACAO_PRAZO', 'TRANCAMENTO_DISCIPLINA', 'EMISSAO_DOCUMENTO'),
    descricao VARCHAR(250) NOT NULL,
    status VARCHAR(20) NOT NULL (DEFAULT 'EM_ANALISE', CHECK: 'EM_ANALISE', 'DEFERIDO', 'INDEFERIDO'),
    data_solicitacao DATETIME2 NOT NULL (DEFAULT GETDATE()),
    aluno_id INT (FK -> PESSOAS.id),
    analisado_por_id INT (FK -> PESSOAS.id)
)

AVISOS (
    id INT (PK, IDENTITY),
    titulo VARCHAR(150) NOT NULL,
    conteudo TEXT NOT NULL,
    data_publicacao DATETIME2 NOT NULL (DEFAULT GETDATE()),
    criado_por_id INT (FK -> PESSOAS.id)
)
```

7. Modelo Físico: Scripts DDL (Criação de Tabelas)

Abaixo está o script SQL completo estruturado para execução no **Microsoft SQL Server**:

```
-- 1. Criar o Banco de Dados
CREATE DATABASE AVAFICR;
GO
USE AVAFICR;
GO

-- 2. Tabela Pessoas (contendo UNIQUE, NOT NULL e CHECK)
CREATE TABLE Pessoas (
    id INT IDENTITY(1,1) PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    cpf VARCHAR(14) NOT NULL UNIQUE,
    senha VARCHAR(100) NOT NULL,
    tipo_pessoa VARCHAR(20) NOT NULL CONSTRAINT chk_tipo_pessoa CHECK (tipo_pessoa IN ('ALUNO', 'PROFESSOR', 'SECRETARIA'))
);
GO
```

```

-- 3. Tabela Disciplinas
CREATE TABLE Disciplinas (
    id INT IDENTITY(1,1) PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    professor_id INT NOT NULL,
    CONSTRAINT fk_disciplinas_professor FOREIGN KEY (professor_id) REFERENCES Pessoas(id)
);
GO

-- 4. Tabela de Associação N:N (DisciplinaAlunos)
CREATE TABLE DisciplinaAlunos (
    disciplina_id INT NOT NULL,
    aluno_id INT NOT NULL,
    CONSTRAINT pk_disciplina_alunos PRIMARY KEY (disciplina_id, aluno_id),
    CONSTRAINT fk_disciplina_alunos_disc FOREIGN KEY (disciplina_id) REFERENCES Disciplinas(id),
    CONSTRAINT fk_disciplina_alunos_aluno FOREIGN KEY (aluno_id) REFERENCES Pessoas(id)
);
GO

-- 5. Tabela Tarefas
CREATE TABLE Tarefas (
    id INT IDENTITY(1,1) PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    descricao TEXT NULL,
    data_entrega DATETIME2 NOT NULL,
    pontos_maximos DECIMAL(5,2) NOT NULL CONSTRAINT chk_pontos_maximos CHECK (pontos_maximos >= 0),
    disciplina_id INT NOT NULL,
    criado_por_id INT NOT NULL,
    CONSTRAINT fk_tarefas_disciplina FOREIGN KEY (disciplina_id) REFERENCES Disciplinas(id),
    CONSTRAINT fk_tarefas_criador FOREIGN KEY (criado_por_id) REFERENCES Pessoas(id)
);
GO

-- 6. Tabela EntregaTarefas (Com EXCLUSÃO EM CASCATA e DEFAULT)
CREATE TABLE EntregaTarefas (
    id INT IDENTITY(1,1) PRIMARY KEY,
    tarefa_id INT NOT NULL,
    aluno_id INT NOT NULL,
    status_tarefa VARCHAR(20) NOT NULL DEFAULT 'PENDENTE' CONSTRAINT chk_status_tarefa CHECK (status_tarefa IN ('PENDENTE', 'EM_ANDAMENTO', 'ENTREGUE')),
    data_entrega DATETIME2 NULL,
    resposta_text TEXT NULL,
    nota DECIMAL(4,2) NULL CONSTRAINT chk_nota CHECK (nota >= 0.0),
    feedback VARCHAR(250) NULL,
    CONSTRAINT fk_entregas_tarefa FOREIGN KEY (tarefa_id) REFERENCES Tarefas(id) ON DELETE CASCADE, -- Cascading Delete
    CONSTRAINT fk_entregas_aluno FOREIGN KEY (aluno_id) REFERENCES Pessoas(id)
);
GO

-- 7. Tabela Protocolos (Com AUTONUMERAÇÃO INICIANDO EM 1000 incrementando em 10)
CREATE TABLE Protocolos (
    id INT IDENTITY(1000, 10) PRIMARY KEY, -- IDENTITY(start, increment)
    tipo VARCHAR(50) NOT NULL CONSTRAINT chk_tipo_protocolo CHECK (tipo IN ('PRORROGACAO_PRAZO', 'TRANCAMENTO_DISCIPLINA', 'EMISSAO_DOCUMENTO')),
    descricao VARCHAR(250) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'EM_ANALISE' CONSTRAINT chk_status_protocolo CHECK (status IN ('EM_ANALISE', 'DEFERIDO', 'INDEFERIDO')),
    data_solicitacao DATETIME2 NOT NULL DEFAULT GETDATE(), -- DEFAULT
    aluno_id INT NOT NULL,
    analisado_por_id INT NULL,
    CONSTRAINT fk_protocolos_aluno FOREIGN KEY (aluno_id) REFERENCES Pessoas(id),
    CONSTRAINT fk_protocolos_analista FOREIGN KEY (analisado_por_id) REFERENCES Pessoas(id)
);
GO

-- 8. Tabela Avisos
CREATE TABLE Avisos (
    id INT IDENTITY(1,1) PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    conteudo TEXT NOT NULL,
    data_publicacao DATETIME2 NOT NULL DEFAULT GETDATE(),
    criado_por_id INT NOT NULL,
    CONSTRAINT fk_avisos_criador FOREIGN KEY (criado_por_id) REFERENCES Pessoas(id)
);
GO

```

8. Alimentação das Tabelas: Scripts DML (Inserção de Dados)

Inserção de **3 registros** em cada tabela criada:

```

-- 1. Inserir Pessoas (Alunos, Professores e Secretaria)
INSERT INTO Pessoas (nome, email, cpf, senha, tipo_pessoa) VALUES
('Thiago Ventura', 'thiago@fucr.edu.br', '111.111.111-11', '123456', 'ALUNO'),
('Wallace Felipe', 'wallace@fucr.edu.br', '222.222.222-22', '123456', 'PROFESSOR'),
('Maria Clara', 'secretaria@fucr.edu.br', '333.333.333-33', '123456', 'SECRETARIA'),
('Jose Gomes', 'jose@fucr.edu.br', '444.444.444-44', '123456', 'PROFESSOR'),
('Ana Costa', 'ana@fucr.edu.br', '555.555.555-55', '123456', 'ALUNO');
GO

-- 2. Inserir Disciplinas
INSERT INTO Disciplinas (nome, professor_id) VALUES
('Desenvolvimento de Sistemas', 2), -- Wallace Felipe
('Banco de Dados II', 4), -- Jose Gomes
('Programação Orientada a Objetos', 2); -- Wallace Felipe
GO

-- 3. Inserir Matrículas na DisciplinaAlunos (N:N)

```

```
INSERT INTO DisciplinaAlunos (disciplina_id, aluno_id) VALUES
(1, 1), -- Thiago em Desenvolvimento de Sistemas
(2, 1), -- Thiago em Banco de Dados II
(1, 5); -- Ana em Desenvolvimento de Sistemas
GO

-- 4. Inserir Tarefas
INSERT INTO Tarefas (titulo, descricao, data_entrega, pontos_maximos, disciplina_id, criado_por_id) VALUES
('CRUD em Spring Boot', 'Criação de endpoints REST.', '2026-06-25 23:59:00', 10.0, 1, 2),
('Mapeamento ORM JPA', 'Relacionamentos entre entidades SQL.', '2026-06-28 23:59:00', 10.0, 2, 4),
('Atividade Prática Docker', 'Configurar containers.', '2026-06-30 23:59:00', 10.0, 1, 2);
GO

-- 5. Inserir Entregas
INSERT INTO EntregaTarefas (tarefa_id, aluno_id, status_tarefa, data_entrega, resposta_text, nota, feedback) VALUES
(1, 1, 'ENTREGUE', '2026-06-18 10:00:00', 'github.com/thiago/crud-spring', 8.5, 'Excelente código estruturado.'),
(2, 1, 'PENDENTE', NULL, NULL, NULL, NULL),
(1, 5, 'ENTREGUE', '2026-06-18 11:30:00', 'github.com/ana/crud-spring', 9.0, 'Parabéns, ótima organização.');
```

```
GO

-- 6. Inserir Protocolos (Gera IDs: 1000, 1010, 1020)
INSERT INTO Protocolos (tipo, descricao, status, aluno_id, analisado_por_id) VALUES
('PRORROGACAO_PRAZO', 'Solicito extensão de prazo para tarefa de Docker.', 'EM_ANALISE', 1, NULL),
('TRANCAMENTO_DISCIPLINA', 'Trancamento devido a conflitos de horários de trabalho.', 'DEFERIDO', 1, 3),
('EMISSAO_DOCUMENTO', 'Necessito de Declaração de Matrícula para estágio.', 'EM_ANALISE', 5, NULL);
GO

-- 7. Inserir Avisos
INSERT INTO Avisos (titulo, conteudo, criado_por_id) VALUES
('Manutenção do Portal', 'Mural indisponível dia 20/06 de 02:00 às 06:00.', 3),
('Início de Matrículas', 'Matrículas para 2026.2 começam amanhã.', 3),
('Feriado Acadêmico', 'Não haverá expediente letivo no dia 25/06.', 3);
GO
```

9. Consultas SQL (DML)

As consultas abaixo atendem aos requisitos de usar apelidos (alias), múltiplos joins e funções de agregação:

Consulta A: Relatório de Notas dos Alunos por Disciplina (2 JOINS + ALIAS)

Retorna o nome do Aluno, a Disciplina, o título da tarefa entregue, a nota obtida e o feedback:

```
SELECT
    a.nome AS Aluno,
    d.nome AS Disciplina,
    t.titulo AS Atividade,
    et.nota AS NotaObtida,
    et.feedback AS Observacoes
FROM EntregaTarefas et
INNER JOIN Pessoas a ON et.aluno_id = a.id
INNER JOIN Tarefas t ON et.tarefa_id = t.id
INNER JOIN Disciplinas d ON t.disciplina_id = d.id
WHERE et.status_tarefa = 'ENTREGUE';
```

Consulta B: Métricas e Médias Acadêmicas (Funções de Agregação e Agrupamento)

Retorna dados agregados de notas (total de pontos acumulados, média de notas, maior nota, menor nota e total de tarefas avaliadas) para cada aluno em cada matéria:

```
SELECT
    a.nome AS Aluno,
    d.nome AS Disciplina,
    COUNT(et.id) AS TotalTarefasAvaliadas,
    SUM(et.nota) AS TotalPontosAcumulados,
    AVG(et.nota) AS MediaNotas,
    MAX(et.nota) AS MaiorNotaObtida,
    MIN(et.nota) AS MenorNotaObtida
FROM EntregaTarefas et
INNER JOIN Pessoas a ON et.aluno_id = a.id
INNER JOIN Tarefas t ON et.tarefa_id = t.id
INNER JOIN Disciplinas d ON t.disciplina_id = d.id
WHERE et.nota IS NOT NULL
GROUP BY a.nome, d.nome;
```

10. Dicionário de Dados

Abaixo está o Dicionário de Dados especificando cada tabela do banco de dados, seguindo exatamente o modelo da instituição:

Tabela: Pessoas

- **Tabela:** Pessoas
- **Descrição:** Tabela destinada ao cadastramento de pessoas físicas usuárias do sistema (Alunos, Professores e membros da Secretaria Acadêmica).
- **Observações:** O tipo de pessoa define as permissões de acesso às funcionalidades do sistema (RBAC).

CAMPOS

Nome da Coluna	Tipo de Dado	Restrições	Descrição	Tabela Relacionada
id	INT	PK IDENTITY(1,1)	Código identificador autoincrementado da pessoa.	Disciplinas(professor_id) DisciplinaAlunos(aluno_id) Tarefas(criado_por_id) EntregaTarefas(aluno_id) Protocolos(aluno_id) Protocolos(analisado_por_id) Avisos(criado_por_id)
nome	VARCHAR(150)	NOT NULL	Nome completo da pessoa.	-
email	VARCHAR(100)	NOT NULL UNIQUE	E-mail de login institucional do usuário.	-
cpf	VARCHAR(14)	NOT NULL UNIQUE	Cadastro de Pessoa Física do usuário.	-
senha	VARCHAR(100)	NOT NULL	Senha de login no portal (armazenada de forma segura).	-
tipo_pessoa	VARCHAR(20)	NOT NULL CHECK (tipo_pessoa IN (‘ALUNO’, ‘PROFESSOR’, ‘SECRETARIA’))	Tipo de ator do sistema.	-

Tabela: Disciplinas

- **Tabela:** Disciplinas
- **Descrição:** Tabela para registro das matérias/disciplinas ofertadas pela instituição.
- **Observações:** Cada disciplina é vinculada a um professor encarregado de ministrá-la.

CAMPOS

Nome da Coluna	Tipo de Dado	Restrições	Descrição	Tabela Relacionada
id	INT	PK IDENTITY(1,1)	Código identificador único da disciplina.	DisciplinaAlunos(disciplina_id) Tarefas(disciplina_id)
nome	VARCHAR(100)	NOT NULL	Nome descritivo da disciplina curricular.	-
professor_id	INT	FK NOT NULL	Código da pessoa (professor) responsável pela matéria.	Pessoas(id)

Tabela: DisciplinaAlunos

- **Tabela:** DisciplinaAlunos
- **Descrição:** Tabela de associação (Junction Table N:N) para registrar a matrícula de alunos nas disciplinas.
- **Observações:** A chave primária é composta de forma conjugada pelas duas chaves estrangeiras.

CAMPOS

Nome da Coluna	Tipo de Dado	Restrições	Descrição	Tabela Relacionada
disciplina_id	INT	PK FK NOT NULL	Código identificador da disciplina matriculada.	Disciplinas(id)
aluno_id	INT	PK FK NOT NULL	Código identificador do aluno matriculado.	Pessoas(id)

Tabela: Tarefas

- **Tabela:** Tarefas
- **Descrição:** Cadastro de atividades e avaliações lançadas para uma disciplina específica.
- **Observações:** A inserção de registros gera automaticamente instâncias de entregas na tabela EntregaTarefas via trigger.

CAMPOS

Nome da Coluna	Tipo de Dado	Restrições	Descrição	Tabela Relacionada
id	INT	PK IDENTITY(1,1)	Código identificador da atividade.	EntregaTarefas(tarefa_id)
titulo	VARCHAR(150)	NOT NULL	Título de identificação da tarefa acadêmica.	-
descricao	TEXT	NULL	Enunciado, regras e orientações da tarefa.	-
data_entrega	DATETIME2	NOT NULL	Data e hora limite para envio de respostas.	-
pontos_maximos	DECIMAL(5,2)	NOT NULL CHECK (pontos_maximos >= 0)	Nota máxima que o aluno pode alcançar.	-
disciplina_id	INT	FK NOT NULL	ID da disciplina vinculada à atividade.	Disciplinas(id)
criado_por_id	INT	FK NOT NULL	Código do professor/membro criador da tarefa.	Pessoas(id)

Tabela: EntregaTarefas

- **Tabela:** EntregaTarefas
- **Descrição:** Tabela que gerencia o estado do Kanban e o controle de notas individuais em cada atividade.
- **Observações:** A exclusão de uma tarefa se propaga na deleção das entregas correspondentes (ON DELETE CASCADE).

CAMPOS

Nome da Coluna	Tipo de Dado	Restrições	Descrição	Tabela Relacionada
id	INT	PK IDENTITY(1,1)	Código identificador único do Kanban de entrega.	-
tarefa_id	INT	FK NOT NULL ON DELETE CASCADE	ID da atividade associada.	Tarefas(id)
aluno_id	INT	FK NOT NULL	ID do aluno a quem pertence a atividade.	Pessoas(id)
status_tarefa	VARCHAR(20)	NOT NULL DEFAULT 'PENDENTE' CHECK (status_tarefa IN ('PENDENTE', 'EM_ANDAMENTO', 'ENTREGUE'))	Status atual da entrega.	-
data_entrega	DATETIME2	NULL	Registro de data e horário da entrega do aluno.	-
resposta_text	TEXT	NULL	Resposta anexada pelo aluno (URLs ou texto).	-
nota	DECIMAL(4,2)	NULL CHECK (nota >= 0.0)	Nota individual atribuída após avaliação.	-
feedback	VARCHAR(250)	NULL	Considerações e feedback do professor.	-

Tabela: Protocolos

- **Tabela:** Protocolos
- **Descrição:** Registro de requerimentos acadêmicos abertos por alunos do portal.
- **Observações:** Numeração de protocolos inicia-se em 1000 com incrementos de 10 em 10 (IDENTITY(1000, 10)).

CAMPOS

Nome da Coluna	Tipo de Dado	Restrições	Descrição	Tabela Relacionada
id	INT	PK IDENTITY(1000,10)	Identificador sequencial especial do chamado.	-
tipo	VARCHAR(50)	NOT NULL CHECK (tipo IN ('PRORROGACAO_PRAZO', 'TRANCAMENTO_DISCIPLINA', 'EMISSAO_DOCUMENTO'))	Tipo de chamado administrativo.	-
descricao	VARCHAR(250)	NOT NULL	Justificativa do pedido feita pelo aluno.	-
status	VARCHAR(20)	NOT NULL DEFAULT 'EM_ANALISE' CHECK (status IN ('EM_ANALISE', 'DEFERIDO', 'INDEFERIDO'))	Status de processamento do chamado.	-
data_solicitacao	DATETIME2	NOT NULL DEFAULT GETDATE()	Data e hora em que a solicitação foi efetuada.	-
aluno_id	INT	FK NOT NULL	ID do aluno que registrou a requisição.	Pessoas(id)
analizado_por_id	INT	FK NULL	ID do funcionário administrativo que avaliou.	Pessoas(id)

Tabela: Avisos

- **Tabela:** Avisos
- **Descrição:** Registro de informes, comunicados gerais e mural de avisos da instituição.
- **Observações:** Visíveis no painel de avisos geral de todos os perfis.

CAMPOS

Nome da Coluna	Tipo de Dado	Restrições	Descrição	Tabela Relacionada
id	INT	PK IDENTITY(1,1)	Código identificador único do aviso.	-

Nome da Coluna	Tipo de Dado	Restrições	Descrição	Tabela Relacionada
titulo	VARCHAR(150)	NOT NULL	Título do comunicado no painel acadêmico.	-
conteudo	TEXT	NOT NULL	Texto principal com o corpo do aviso.	-
data_publicacao	DATETIME2	NOT NULL DEFAULT GETDATE()	Data de postagem oficial do comunicado.	-
criado_por_id	INT	FK NOT NULL	ID do secretário/administrador autor do aviso.	Pessoas(id)

11. Automação e Desempenho (Triggers, Functions, Procedures e Index)

Implementações procedurais avançadas no Microsoft SQL Server (T-SQL) para cobrir as regras de negócio do AVA:

A. TRIGGER (Geração Automática do Quadro de Atividades)

Regra de negócio: Sempre que um professor cadastrar uma nova tarefa, esse Trigger intercepta a inserção e gera automaticamente um registro de entrega com status 'PENDENTE' para cada um dos alunos matriculados na respectiva disciplina.

```
CREATE TRIGGER trg_GerarEntregasTarefa
ON Tarefas
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    -- Insere registros de entrega no Kanban dos alunos matriculados na disciplina da tarefa criada
    INSERT INTO EntregaTarefas (tarefa_id, aluno_id, status_tarefa, data_entrega, resposta_text, nota, feedback)
    SELECT
        i.id,
        da.aluno_id,
        'PENDENTE',
        NULL,
        NULL,
        NULL,
        NULL
    FROM inserted i
    INNER JOIN DisciplinaAlunos da ON da.disciplina_id = i.disciplina_id;
END;
GO
```

B. FUNCTION (Cálculo de Média de Disciplina)

Regra de negócio: Recebe o ID do Aluno e o ID da Disciplina e calcula a média ponderada/aritmética simples de todas as tarefas já avaliadas e pontuadas pelo professor naquela disciplina.

```
CREATE FUNCTION fn_CalcularMediaAluno (
    @aluno_id INT,
    @disciplina_id INT
)
RETURNS DECIMAL(4, 2)
AS
BEGIN
    DECLARE @media DECIMAL(4, 2);

    -- Calcula a média aritmética simples das notas de tarefas avaliadas
    SELECT @media = AVG(et.nota)
    FROM EntregaTarefas et
    INNER JOIN Tarefas t ON t.id = et.tarefa_id
    WHERE et.aluno_id = @aluno_id
        AND t.disciplina_id = @disciplina_id
        AND et.nota IS NOT NULL;

    -- Retorna 0.0 caso não existam notas lançadas
    RETURN ISNULL(@media, 0.0);
END;
GO
```

C. STORED PROCEDURE (Matrícula e Sincronização retroativa de Atividades)

Regra de negócio: Realiza a matrícula de um aluno em uma disciplina. Valida se o aluno já está matriculado (evitando duplicidade). Caso a disciplina já possua tarefas criadas anteriormente, a procedure insere retroativamente as entregas 'PENDENTES' no quadro do aluno recém-matriculado.

```
CREATE PROCEDURE sp_MatricularAluno
    @aluno_id INT,
    @disciplina_id INT
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Validação de Duplicidade
    IF EXISTS (SELECT 1 FROM DisciplinaAlunos WHERE aluno_id = @aluno_id AND disciplina_id = @disciplina_id)
    BEGIN
        RAISERROR('Erro: Aluno já se encontra matriculado nesta disciplina.', 16, 1);
        RETURN;
    END;

    -- 2. Inserir registro de matrícula
    INSERT INTO DisciplinaAlunos (aluno_id, disciplina_id)
    VALUES (@aluno_id, @disciplina_id);
END;
```

18/06/2026, 22:57

```
-- 2. Inserir Matrícula
INSERT INTO DisciplinaAlunos (disciplina_id, aluno_id)
VALUES (@disciplina_id, @aluno_id);

-- 3. Geração Retroativa de Tarefas pendentes no Kanban do aluno
INSERT INTO EntregaTarefas (tarefa_id, aluno_id, status_tarefa, data_entrega, resposta_text, nota, feedback)
SELECT
    t.id,
    @aluno_id,
    'PENDENTE',
    NULL,
    NULL,
    NULL,
    NULL
FROM Tarefas t
WHERE t.disciplina_id = @disciplina_id;

PRINT 'Aluno matriculado e atividades sincronizadas com sucesso!';
END;
GO
```

D. INDEX (Otimização de Busca Textual por Nome)

Regra de negócio: Otimizar o desempenho do banco em consultas alfabéticas pelo nome da pessoa (utilizado na barra de pesquisa da secretaria e no login).

```
CREATE INDEX idx_Pessoas_Nome
ON Pessoas (nome);
GO
```